embedding, feed it through our adapter (a small MLP), and produce vectors in the LLM's input embedding dimension. We then prompt the LLM with these "vision tokens" and train it to generate the image's caption. By doing this, the LLM learns to "read" visual features as if they were text tokens, bridging the modalities without touching most of the model weights.

Process Outline:

- **Model and Data:** Choose a compact LLM and image encoder. For example, use Gemma-2 (2B) or Phi-2 as the LLM [8], and a SigLIP image model (400M or 1B) [11]. Collect image–text pairs for training (e.g. MS-COCO captions, Conceptual Captions, or LAION alt-text).
- **Adapter Design:** Build a small projection network (e.g. a two-layer MLP with nonlinearity) that takes the image embedding and outputs a tensor in $\mathbb{R}^{T \times d}$ (as if T tokens of dimension d, or even a single token of dimension d). One strategy (as in many VLMs) is to project to a fixed number of "image tokens" that prepend the text. Alternatively, project into one token's embedding and prepend a special image token. We will experiment with the simplest: a linear or MLP adapter that outputs *one* LLM token embedding for the entire image [12].
- **Training Stage 1:** Freeze the vision encoder and LLM. Train only the adapter on image–caption pairs (cross-entropy loss to predict the caption text). This aligns the image embedding to the LLM's text space. (This mirrors LLaVA's first stage: optimize the vision-to-language adapter on captions [13].) The adapter learns to produce embedding vectors that make sense to the LLM.
- **Training Stage 2 (optional):** We can optionally unfreeze and fine-tune part of the LLM on a smaller instruction-tuning dataset to improve multimodal understanding, but the core idea is that Stage 1 already enables the model to *see* images.
- **Evaluation:** Test the resulting model on image captioning and simple VQA. For example, feed an image, generate a caption, and compute BLEU or CLIPScore. Also try a few visual dialog prompts (e.g. "What is in this image?"). The goal is not SOTA multimodal performance but demonstrating that the LLM can interpret the projected image embedding.

Pre-reads and Resources: We will draw on recent multimodal adapter literature. LLaVA's methodology shows that a two-stage train (adapter-only then instruction tune) works well [13]. In fact, LLaVA-MORE notes: *"In the first stage, only the vision-to-language adapter is optimized to align the image features with the text embedding space"* [13]. Hugging Face blogs and tutorials confirm that one can train small VLMs on Colab by freezing main models and learning a connector [12]. The blog *"Vision-language models from scratch in Colab"* explicitly describes taking frozen vision and text models and training a projection so that image embeddings are mapped into the LLM's token embedding space [12]. We will also consult the surveys of small VLMs, the CLIP/SigLIP papers (which emphasize the power of their embeddings [10]), and recent work on embedding alignment. A helpful example is *ProCLIP* [14], which distills CLIP's textual knowledge into an LLM embedding (similar in spirit to our vision→text mapping). These references will guide architecture choices and training protocols.

Feasibility and Value: Training only a small adapter (<<100M parameters) is feasible on modest hardware (even a GPU-enabled Colab) [12]. We avoid full multimodal finetuning, making this project lightweight. The adapter can train in ~10–20 hours on standard image–caption datasets. For a CV, this is a solid systems+ML project: it demonstrates understanding of multimodal model design, efficient training (only adapter), and evaluation. It's similar to what open-source projects (LLaVA, BLIP-2) do, but scaled down and with clear ablations. If the adapter successfully enables the LLM to generate reasonable captions, that's a compelling

result. It shows knowledge of embedding space alignment—a trendy research topic (e.g. ProCLIP's curriculum for CLIP↔LLM) and an active area of inquiry.

Technical Plan:

- **Setup:** Load a pretrained LLM (e.g. Gemma-2 2B [8]) and vision encoder (e.g. SigLIP-ViT-Small [11]) from Hugging Face. Freeze all LLM and vision weights.
- **Adapter Module:** Implement an adapter (e.g. `nn.Linear(vision_dim, lm_dim)` or a small MLP). Option A: Map the image embedding to one LLM token embedding (so we prepend a special [IMG] token whose embedding is this output). Option B: Split the image embedding into several tokens. Normalize and project as in [38] (they use a layernorm and two linear layers) [12].
- **Data Pipeline:** Use image–caption datasets. Pre-tokenize captions for the LLM. For each (image, caption), compute the image embedding, run it through the adapter to produce `input_embeds`, then run the LLM to predict the caption tokens.
- **Training Loop:** As [38] suggests, train for one epoch of image-caption pairs: minimize cross-entropy loss on the caption, backprop through the adapter. Keep the rest of the model frozen. This aligns vision→text embedding. (We may use teacher forcing: feed correct caption tokens as input during training after the image token.)
- **Stage 2 Tuning (Optional):** If needed, we can unfreeze the last few layers of the LLM and train on a small multimodal instruction dataset (like LLaVA's) to refine the image understanding. But we expect Stage 1 to already give the LLM some basic ability.
- **Evaluation:** Test on held-out image-caption pairs. Prompt the model with an image (via our adapter) and measure caption quality (BLEU, METEOR, or CLIPScore). Also ask a few qualitative questions (e.g. "Describe this image.") to see if the LLM generates coherent answers. Compare to a baseline (e.g. caption-only LLM or original vision model).
- **Analysis:** Inspect adapter outputs: do semantically similar images map to similar token embeddings? Perhaps probe by nearest neighbors in embedding space. Also visualize how well the model attends to the "image token" when generating.
- **Ablations and Documentation:** Vary adapter size and architecture (one-layer vs two-layer, with or without nonlinearity). Possibly try different training data (COCO vs LAION). Document all findings.

By following this plan—grounded in examples from LLaVA and Colab tutorials [13] [12]—we can feasibly build a small multimodal LLM. A successful result (an LLM reading images via distilled embeddings) would be a clear demonstration of research acumen in multimodal AI.

Sources: We draw on recent research and guides for both projects. For Project 1, the *QuickSilver* paper introduces per-token quantization and reports large speedups with minimal quality loss [2] [3]. For Project 2, guides like Nethercott's Colab tutorial show practical adapter training [12] [15], and LLaVA/MORE provide the conceptual foundation of vision-to-language adapters [13] [9]. These and other references (cited above) form the supporting literature for our plans.

---

[1] [2] [3] [6] [7] QuickSilver -- Speeding up LLM Inference through Dynamic Token Halting, KV Skipping, Contextual Token Fusion, and Adaptive Matryoshka Quantization
https://arxiv.org/pdf/2506.22396

[4] Quantization - vLLM
https://docs.vllm.ai/en/latest/features/quantization/

[5] 2:4 Sparse Llama: Smaller models for efficient GPU inference | Red Hat Developer
https://developers.redhat.com/articles/2025/02/28/24-sparse-llama-smaller-models-efficient-gpu-inference

[8] Gemma 2: Improving Open Language Models at a Practical Size
https://arxiv.org/html/2408.00118v1

[9] [13] LLaVA-MORE : A Comparative Study of LLMs and Visual Backbones for Enhanced Visual Instruction Tuning
https://arxiv.org/html/2503.15621v2

[10] [11] arxiv.org
https://arxiv.org/pdf/2502.14786

[12] [15] Vision-language models from scratch in colab | by Nate Nethercott | Medium
https://medium.com/@natenethercott/vision-language-models-from-scratch-in-colab-cd073a753b8a

[14] ProCLIP: Progressive Vision-Language Alignment via LLM-based Embedder
https://arxiv.org/html/2510.18795v3